

Teorie C++: std::array și std::vector

Standard Template library (STL)

STL (Standard Template Library) este o bibliotecă în limbajul de programare C++ care oferă o colecție de șabloane de clase și funcții pentru gestionarea diferitelor structuri de date și operațiuni asociate acestora. STL a fost inclusă în standardul C++ pentru a furniza o serie de instrumente de bază pentru dezvoltatorii C++ în scopul facilitării dezvoltării de cod mai eficient și mai ușor de întreținut.

Principalele componente ale STL includ următoarele:

1. **Containere:** STL furnizează o varietate de containere, precum vectori, liste, cozi, stive, mape (și multe altele) pentru a stoca și organiza datele. Aceste containere sunt implementate sub forma unor șabloane de clase, ceea ce înseamnă că puteți folosi aceste șabloane pentru a crea obiecte de date personalizate.
2. **Iteratori:** Iteratoarele sunt utilizate pentru a parcurge elementele dintr-un container STL. Acestea oferă un mecanism uniform pentru a accesa elementele din oricare container, indiferent de tipul de container.
3. **Algoritmi:** STL furnizează o serie de algoritmi predefiniți care pot fi aplicați pe containerele STL. Acești algoritmi includ sortare, căutare, transformare și multe altele. Acestea sunt utile pentru a efectua operații comune de procesare a datelor într-un mod eficient.
4. **Adaptori:** STL oferă adaptori care permit să transformi o interfață de la un container în alta. De exemplu, puteți utiliza un adaptor pentru a transforma o stivă într-o coadă sau invers.
5. **Funcții funcționale:** Acestea permit să definiți funcții lambda și alte obiecte funcționale care pot fi folosite cu algoritmii STL pentru a personaliza comportamentul acestora.

STL face dezvoltarea în C++ mult mai eficientă, deoarece oferă structuri de date bine testate, algoritmi și mecanisme de acces la date. Dezvoltatorii pot utiliza aceste resurse pentru a scrie cod mai concis și mai ușor de înțeles. STL este o parte fundamentală a limbajului C++ și este utilizată pe scară largă în dezvoltarea de software C++.

Link-uri utile documentatie STL

cppreference.com

cplusplus.com

std::array

Documentatie online: std::array - cppreference.com

std::array este un container fix din Standard Template Library (STL) din limbajul de programare C++. Acesta oferă un tablou de dimensiuni fixe cu toate caracteristicile și avantajele containerelor STL, cum ar fi siguranța la tipare și gestionarea ușoară a elementelor. Dacă știți deja dimensiunea tabloului la momentul compilării și nu doriți să o schimbați, std::array este o alegere bună.

Declararea unui std::array

Sintaxa este: `std::array < tipul_de_date_folosit, dimensiunea > nume;`

```
#include <array>

std::array<int, 5> myArray; // Declara un std::array de int-uri
```

În acest exemplu, am declarat un std::array numit myArray, care conține numere întregi (int) și are o dimensiune de 5. Dimensiunea este specificată ca al doilea argument în șablonul std::array.

Inițializarea unui std::array

```
std::array<int, 5> myArray = {1, 2, 3, 4, 5};
```

Accesarea elementelor într-un std::array

```
int element1 = myArray[0]; // Prima poziție
int element2 = myArray.at(1); // A doua poziție
```

Accesul la elementele unui std::array se face folosind operatorul [] sau metoda .at()

Obținerea dimensiunii unui std::array

```
int size = myArray.size(); // Va fi 5 în acest caz
```

std::vector

Documentatie online: std::vector - cppreference.com

std::vector este una dintre clasele de container din Standard Template Library (STL) a limbajului de programare C++. Acesta este un container dinamic, ceea ce înseamnă că poate crește sau scădea în dimensiune pe măsură ce elemente sunt adăugate sau eliminate din el. std::vector este foarte versatil și ușor de utilizat, fiind alegerea obișnuită pentru stocarea și gestionarea unui șir (sau tablou) de elemente.

Declararea unui std::vector

Sintaxa este: **std::vector < tipul_de_date_folosit > nume (dimensiune_initala);**

```
#include <vector> // Incluziunea header-ului pentru std::vector

// Declarare și inițializare a unui vector de întregi
std::vector<int> myVector;

std::vector<double> doubleVector(10); // Crează un vector cu 10 elemente
```

Funcție de citire

Fara push_back()

```
#include <vector>
#include <iostream>

void citesteVector(std::vector<int>& vec) {
    int n;
    std::cout << "Introduceti dimensiunea vectorului: ";
    std::cin >> n;

    vec.resize(n); // Redimensioneaza vectorul la dimensiunea dorită

    std::cout << "Introduceti " << n << " elemente: ";
    for (int i = 0; i < n; i++) {
        std::cin >> vec[i];
    }
}
```

Cu push_back()

```
#include <iostream>
#include <vector>

void citesteCuPushBack(std::vector<int>& vec) {
    int n;
    std::cout << "Introduceți numărul de elemente: ";
    std::cin >> n;

    // Resizing vector pentru a face loc pentru cele n elemente
    vec.resize(n);

    std::cout << "Introduceți " << n << " elemente: ";
    for (int i = 0; i < n; i++) {
        int element;
        std::cin >> element;

        // Utilizarea push_back pentru a adăuga elementul la sfârșit
        vec.push_back(element);
    }
}
```

Funcție de afișare

```
#include <vector>
#include <iostream>

void afiseazaVector(const std::vector<int>& vec) {
    std::cout << "Elementele vectorului sunt: ";
    for (const int& element : vec) {
        std::cout << element << " ";
    }
    std::cout << std::endl;
}
```

Funcție de inserare al unui element

```
#include <vector>

void insereazaInVector(std::vector<int>& vec, int valoare, int pozitie) {
    if (pozitie >= 0 && pozitie <= vec.size()) {
        vec.insert(vec.begin() + pozitie, valoare);
    } else {
        // Poate afișa un mesaj de eroare sau efectua altă acțiune în funcție de caz.
    }
}
```

Funcție de ștergere al unui element

```
#include <vector>

void stergeDinVector(std::vector<int>& vec, int pozitie) {
    if (pozitie >= 0 && pozitie < vec.size()) {
        vec.erase(vec.begin() + pozitie);
    } else {
        // Poate afișa un mesaj de eroare sau efectua altă acțiune în funcție de caz.
    }
}
```